

SECURITY OPERATIONS CENTER

Sentinel

SOC Dashboard

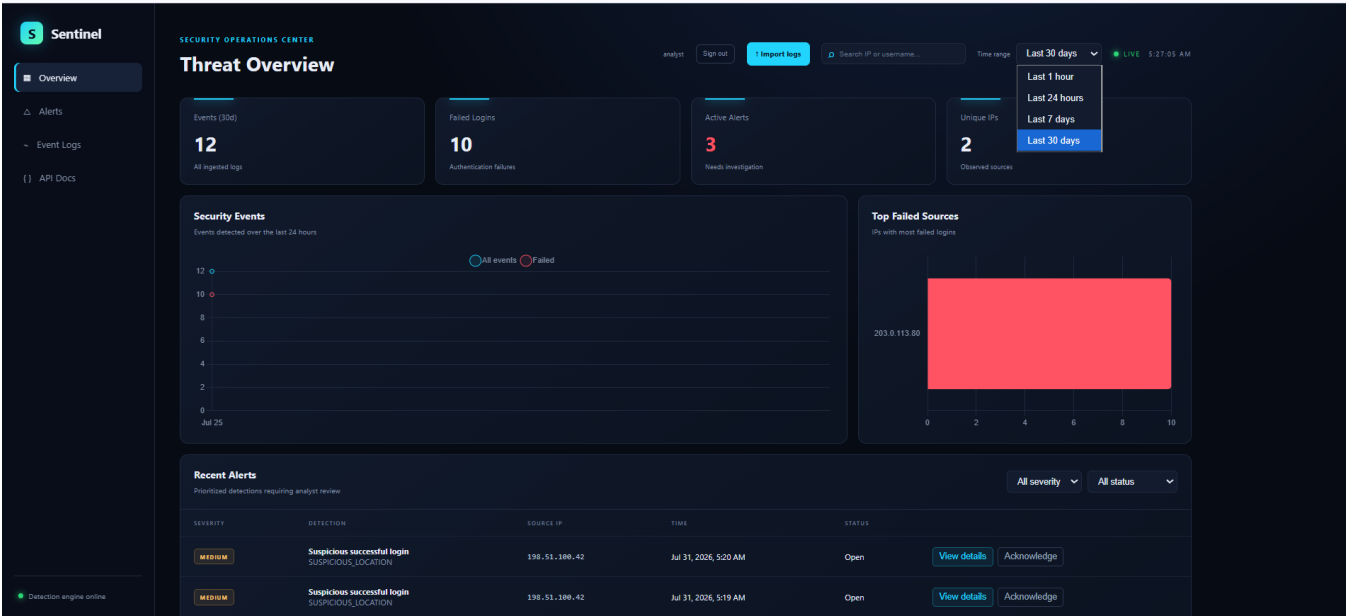
Deployment & Technical Project Documentation

A portfolio-ready mini SIEM for ingesting authentication logs, detecting suspicious login activity, triaging alerts, and documenting security incidents.

AUTHOR
Nahnu Rohmania

STATUS
Production deployed

STACK
FastAPI · PostgreSQL · Vercel



Production overview showing the deployed SOC dashboard and active telemetry.

Prepared as a technical case study for cybersecurity, cloud, and internship portfolios.

Executive summary

Sentinel SOC Dashboard is a defensive security monitoring application that demonstrates an end-to-end Security Operations Center workflow. It accepts structured authentication telemetry, stores events in PostgreSQL, evaluates detection rules, creates analyst-facing alerts, and presents operational metrics through a responsive web dashboard.

OUTCOME

The application was validated locally with automated tests and deployed as a production web service. Its readiness endpoint confirms that both the API and remote PostgreSQL database are available.

Project objectives

- Model the core workflow of a lightweight SIEM: collect → normalize → detect → alert → investigate.
- Visualize failed logins, active alerts, unique source IPs, timelines, and top failed-login sources.
- Provide an analyst workflow for searching, filtering, reviewing, acknowledging, and exporting incidents.
- Apply production-minded controls including hashed credentials, server-side sessions, rate limiting, secret isolation, and health checks.
- Produce a deployable portfolio project with clear evidence of functionality and testing.

Final implementation snapshot

Area	Implementation	Result
Application	FastAPI backend + HTML/CSS/JavaScript dashboard	Deployed
Database	Managed PostgreSQL with encrypted connection	Connected
Detection	Brute-force and suspicious-location rules	Operational
Verification	Automated pytest suite	7 tests passed
Operations	Liveness and readiness endpoints	Healthy

Architecture and data flow

The system separates collection, persistence, detection, presentation, and analyst action. This keeps the project easy to reason about while still reflecting the responsibilities found in a real SOC platform.

Log source	Ingestion API	PostgreSQL	Detection engine	Security alert	SOC analyst
Log source → Ingestion API → PostgreSQL → Detection engine → Security alert → SOC analyst					

Processing sequence

Stage	Responsibility
1. Collect	Accept individual JSON events or bulk CSV/JSON uploads.
2. Validate	Enforce schema, size, field, and time-range constraints.
3. Persist	Write normalized security events to PostgreSQL.
4. Detect	Evaluate failed-login bursts and flagged-location activity.
5. Alert	Create severity-tagged incidents with source-log evidence.
6. Investigate	Display context, filter alerts, acknowledge, and export reports.

Technology stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript, Chart.js	Dashboard, charts, filtering, modals
Backend	Python 3.12, FastAPI	API, validation, auth, orchestration
Data	PostgreSQL, SQLAlchemy	Persistent event and alert storage
Testing	pytest	API and detection regression tests
Deployment	Vercel + managed PostgreSQL	Public hosting and database service
Source control	GitHub	Version history and portfolio visibility

Production deployment and health

Production configuration is supplied through environment variables rather than committed files. The application exposes separate liveness and readiness checks so infrastructure monitoring can distinguish an available process from an application that can successfully reach its database.

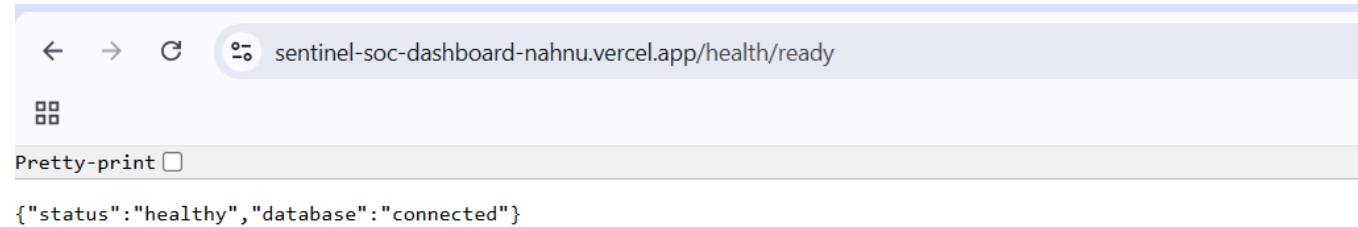


Figure 1. Production readiness check. The API reports a healthy state and confirms the database connection.

INTERPRETATION

`/health/live` confirms the process is running; `/health/ready` additionally verifies database connectivity. This split supports clearer diagnosis during deployment incidents.

Deployment safeguards

- Production mode rejects weak or default authentication secrets.
- Secure cookies are required in production and are transmitted over HTTPS.
- Database credentials, session secrets, and password hashes remain outside the Git repository.
- Connection pooling limits database resource consumption.
- The managed database connection requires TLS.

Log ingestion workflow

Analysts can upload UTF-8 CSV or JSON files directly from the dashboard. The interface enforces a 2 MB limit and a maximum of 1,000 records per upload, then reports imported, rejected, and alert-producing records.

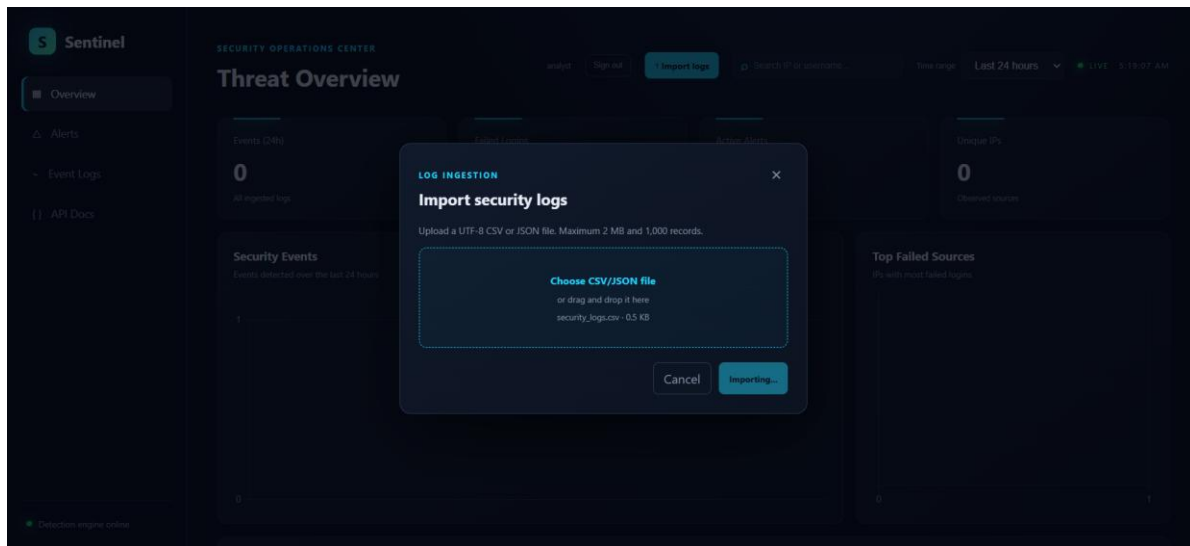


Figure 2. A CSV file selected for ingestion while the dashboard displays a controlled loading state.

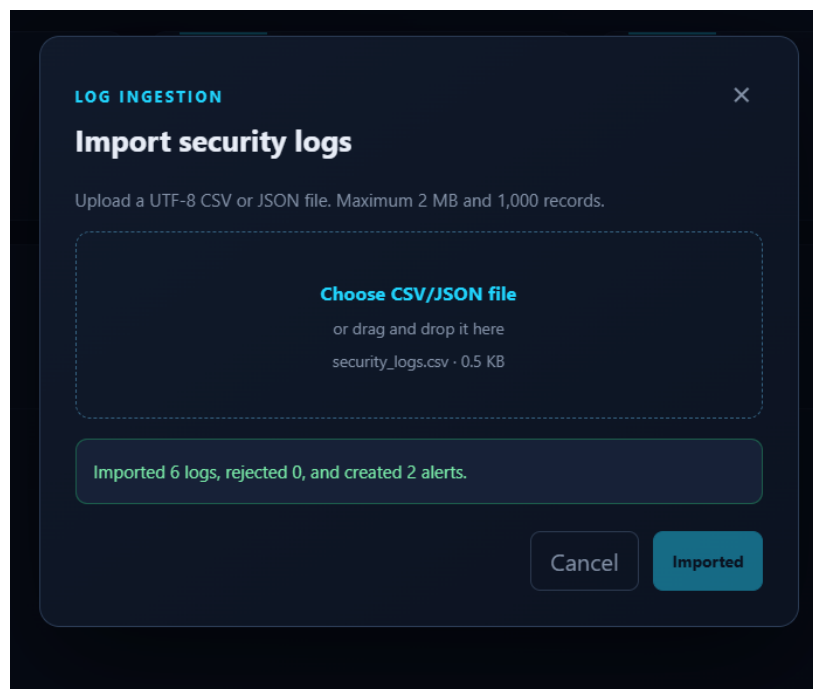


Figure 3. Successful batch processing: 6 records imported, 0 rejected, and 2 alerts generated.

Why this matters

The ingestion result makes the detection pipeline observable. Instead of only confirming that a file was uploaded, the system states how much telemetry was accepted and how many security findings were produced.

Threat overview and operational metrics

The overview consolidates the information an analyst needs for initial triage: event volume, failed authentication attempts, active alerts, unique sources, event trends, and the most persistent failed-login IPs.

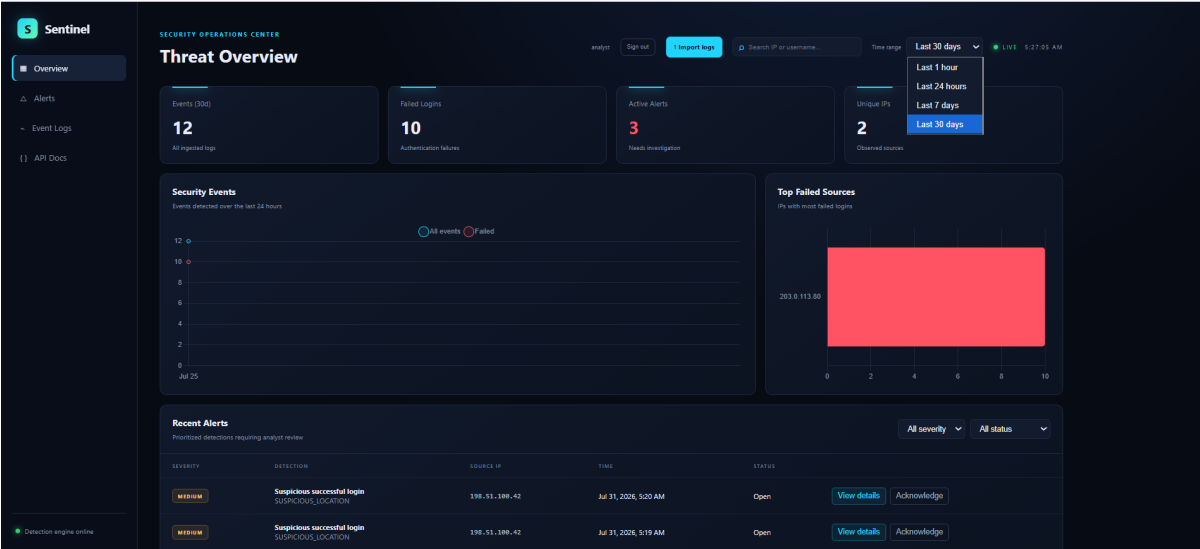


Figure 4. Production dashboard with 30-day scope, KPI cards, timeline visualization, top source, and recent alerts.

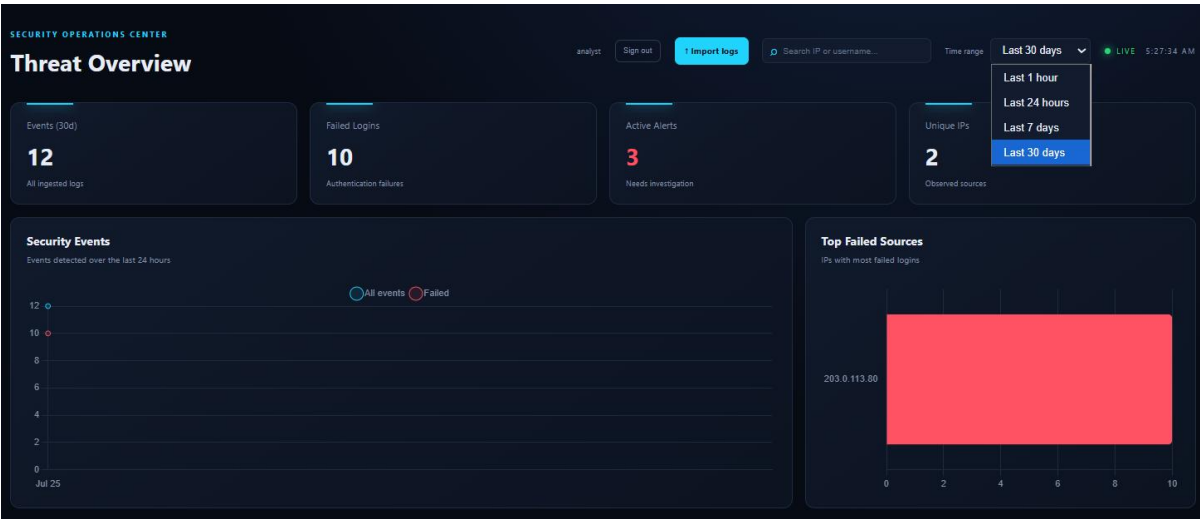


Figure 5. Compact dashboard layout demonstrating responsive presentation of the same monitoring data.

Metric	Operational meaning
Events	Total authentication telemetry inside the selected time window.
Failed Logins	Authentication attempts with a failed result.
Active Alerts	Detections that still require analyst review.
Unique IPs	Distinct source addresses observed in the selected period.
Top Failed Sources	Sources responsible for the most failed logins.

Detection and alert triage

Detection output is presented as an actionable alert queue. Each row contains severity, rule, source, time, status, and analyst controls. The example below shows medium-severity suspicious-location findings alongside a high-severity brute-force detection.

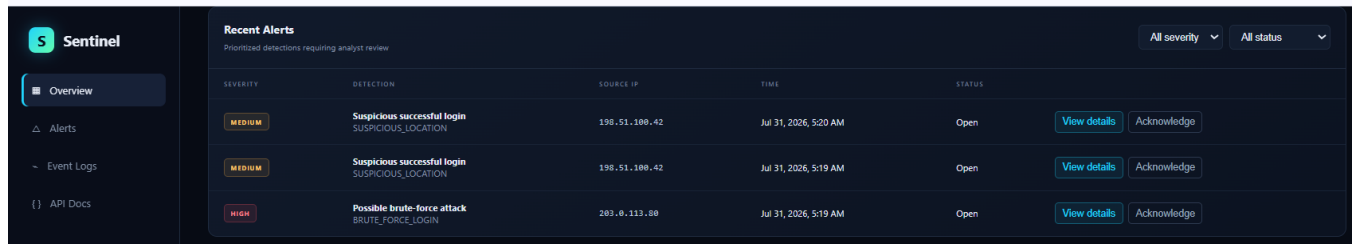


Figure 6. Alert queue with severity classification and separate investigation and acknowledgement actions.

Implemented detection logic

Rule	Trigger	Severity	Analyst value
BRUTE_FORCE_LOGIN	Repeated failed logins from one IP within a defined time window	High	Highlights credential-attack behavior
SUSPICIOUS_LOCATION	Successful login from a flagged or high-risk location	Medium	Surfaces unusual access requiring validation

DESIGN NOTE

The rules are intentionally understandable and testable. They demonstrate detection engineering concepts without pretending to replace a production SIEM or threat-intelligence platform.

Incident investigation and reporting

Selecting an alert opens a structured incident view. The analyst receives the detection rule, source IP, username, event type, result, country classification, timestamp, and raw source event in one place.

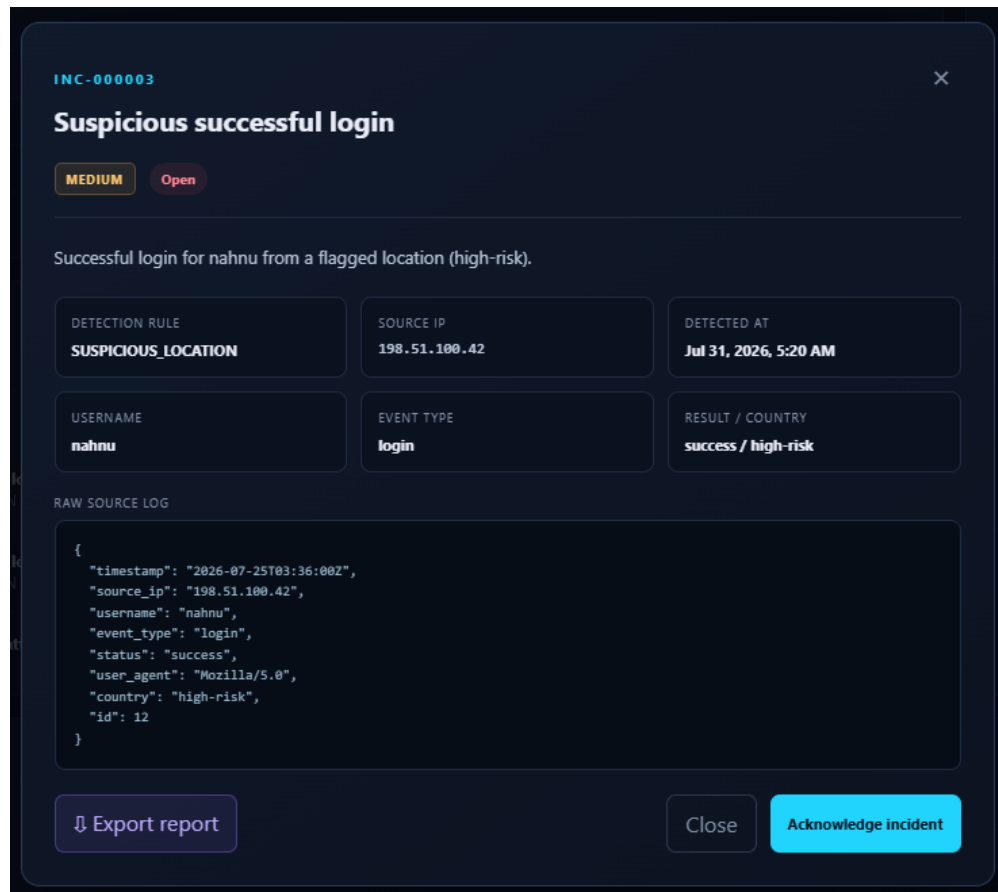


Figure 7. Incident detail panel for a suspicious successful login from a high-risk location.

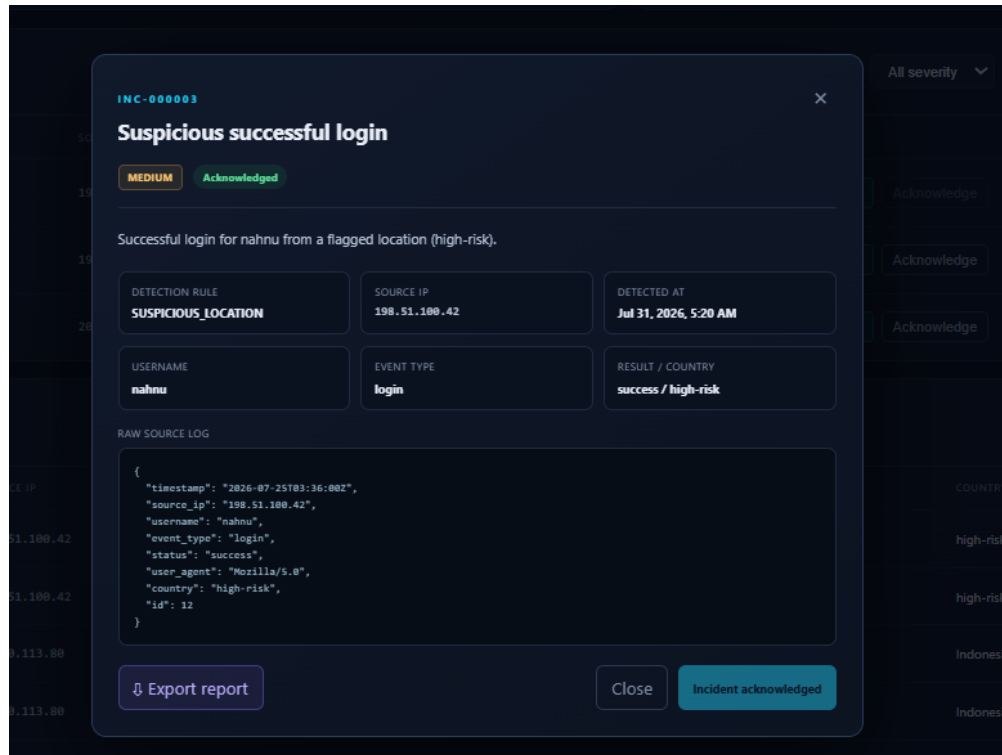


Figure 8. Incident state after acknowledgement, preserving context while recording analyst action.

Incident lifecycle

- **Open:** a new detection is waiting for review.
- **Investigate:** the analyst compares normalized fields with the raw event.
- **Acknowledge:** the analyst marks the finding as reviewed.
- **Export:** the platform generates a portable incident report for evidence or portfolio demonstration.

Exportable incident evidence

The generated report turns an interactive dashboard finding into a standalone artifact. It preserves the incident identifier, detection context, source telemetry, analyst state, and a defensive-use disclaimer.

SENTINEL SECURITY OPERATIONS CENTER

Incident Report

INC-000003
Generated 2026-07-30 22:33 UTC

MEDIUM **OPEN**

Suspicious successful login

Successful login for nahnu from a flagged location (high-risk).

Detection Context

INCIDENT ID INC-000003	DETECTION RULE SUSPICIOUS_LOCATION	DETECTED AT 2026-07-30 22:20:54.116124+00:00
SOURCE IP 198.51.100.42	USERNAME nahnu	EVENT / RESULT login / success
COUNTRY high-risk	USER AGENT Mozilla/5.0	ANALYST STATUS Open

Raw Source Log

```
{
  "id": 12,
  "timestamp": "2026-07-25T03:36:00+00:00",
  "source_ip": "198.51.100.42",
  "username": "nahnu",
  "event_type": "login",
  "status": "success",
  "user_agent": "Mozilla/5.0",
  "country": "high-risk"
}
```

This report was generated by Sentinel SOC Dashboard for defensive security monitoring and incident documentation. Validate findings before taking operational action.

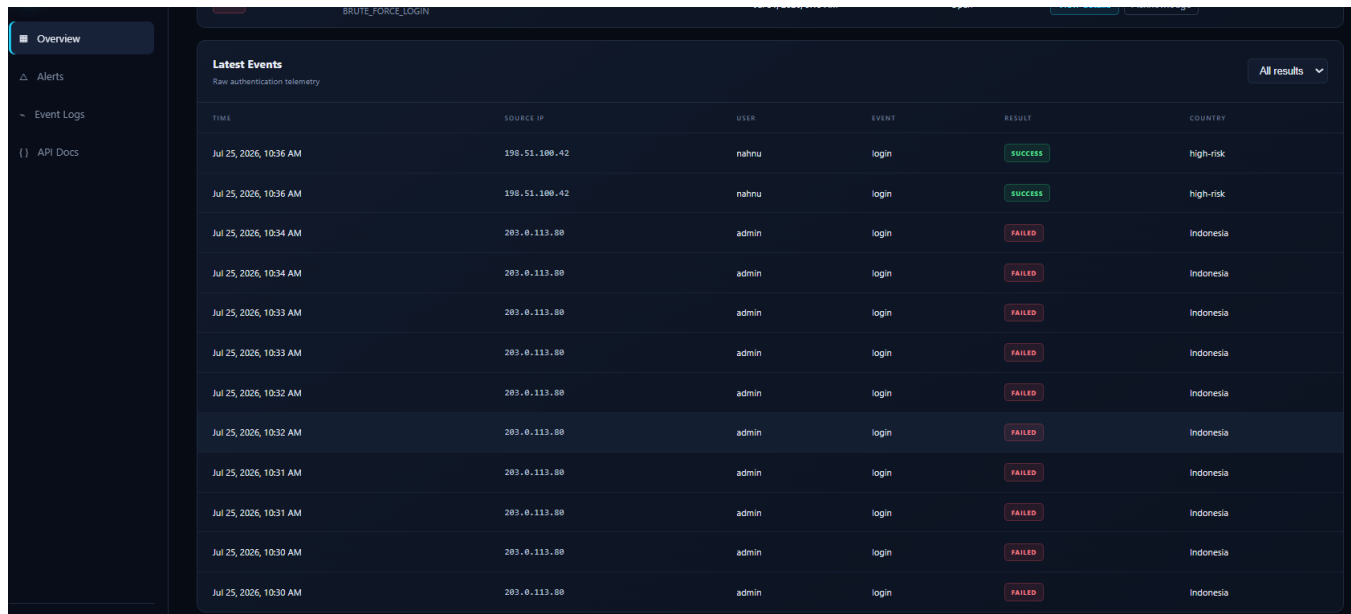
Figure 9. Exported incident report generated from the selected alert.

PORTFOLIO VALUE

This feature demonstrates more than visualization: it shows a full triage workflow and the ability to produce evidence that can be reviewed outside the application.

Raw event visibility

The Latest Events table provides transparent access to the underlying authentication telemetry. Analysts can compare timestamps, IP addresses, users, event types, results, and country values without leaving the dashboard.



TIME	SOURCE IP	USER	EVENT	RESULT	COUNTRY
Jul 25, 2026, 10:36 AM	198.51.100.42	nahu	login	SUCCESS	high-risk
Jul 25, 2026, 10:36 AM	198.51.100.42	nahu	login	SUCCESS	high-risk
Jul 25, 2026, 10:34 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:34 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:33 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:33 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:32 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:32 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:31 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:31 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:30 AM	203.0.113.88	admin	login	FAILED	Indonesia
Jul 25, 2026, 10:30 AM	203.0.113.88	admin	login	FAILED	Indonesia

Figure 10. Raw authentication events showing successful and failed logins used by the detection engine.

Example scenario represented in the evidence

- Repeated failed logins for the `admin` account originate from a documentation-safe IP address.
- The failed sequence crosses the brute-force threshold and creates a high-severity alert.
- Successful logins from a high-risk location create separate suspicious-location alerts.
- The analyst can trace each alert back to its normalized source record.

API surface and developer usability

FastAPI automatically publishes an OpenAPI specification and interactive documentation. This gives developers and reviewers a discoverable contract for ingestion, queries, alerts, reports, statistics, and health checks.

Sentinel SOC Dashboard 1.0.0 OAS 3.1

[/openapi.json](#)

default

POST	/api/logs	Ingest Log	📄	▼
GET	/api/logs	List Logs		▼
POST	/api/logs/upload	Upload Logs		▼
GET	/api/alerts	List Alerts		▼
GET	/api/alerts/{alert_id}	Get Alert Detail		▼
GET	/api/alerts/{alert_id}/report	Incident Report		▼
PATCH	/api/alerts/{alert_id}/acknowledge	Acknowledge Alert		▼
GET	/api/stats/overview	Overview		▼
GET	/api/stats/timeline	Timeline		▼
GET	/api/stats/top-ips	Top Ips		▼
GET	/health	Health		▼
Schemas				▼

Figure 11. Interactive API documentation exposing ingestion, alert, reporting, statistics, and health endpoints.

default

^

POST /api/logs Ingest Log

^

Parameters

Try it out

No parameters

Request body ^{required}

application/json

Example Value | Schema

```
{
  "timestamp": "2026-07-30T22:30:58.559Z",
  "source_ip": "string",
  "username": "string",
  "event_type": "login",
  "status": "string",
  "user_agent": "string",
  "country": "string"
}
```

Responses

Code	Description	Links
201	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "timestamp": "2026-07-30T22:30:58.588Z", "source_ip": "string", "username": "string", "event_type": "login", "status": "string", "user_agent": "string", "country": "string", "id": 0 }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>	No links

GET /api/logs List Logs

^

Figure 12. POST /api/logs contract with request validation and response schemas.

GET /api/logs List Logs

Try it out

Name	Description
limit integer (query)	Default value : 50 50
hours integer (query)	Default value : 24 24
q string (string null) (query)	q
status string (string null) (query)	status

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header: Example Value Schema <pre>{ "timestamp": "2026-07-30T22:31:28.421Z", "source_ip": "string", "username": "string", "event_type": "login", "status": "string", "user_agent": "string", "country": "string", "id": 0 }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>	No links

Figure 13. GET /api/logs query controls for pagination, time range, search, and status filtering.

Data contracts and validation

Schema definitions document the shape of alerts, paginated results, uploads, and log records. Typed contracts reduce ambiguity between the frontend, backend, tests, and any future collector integration.

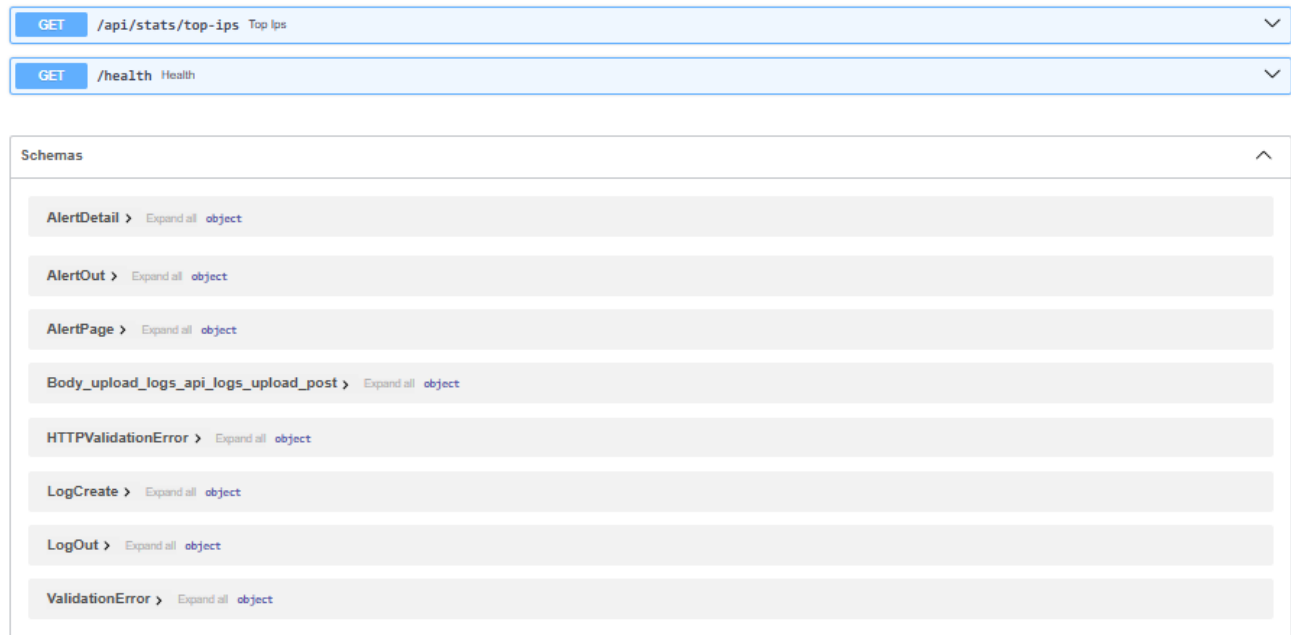


Figure 14. OpenAPI schema catalog for alert, upload, log, and validation objects.



Figure 15. AlertDetail schema, including linked source-log evidence.

AlertOut ^ Collapse all object <pre> id* integer title* string description* string severity* string rule_name* string source_ip* string acknowledged* boolean created_at* string date-time </pre>
AlertPage ^ Collapse all object <pre> items* > Expand all array<object> total* integer page* integer page_size* integer </pre>
Body_upload_logs_api_logs_upload_post ^ Collapse all object <pre> file* string binary </pre>
HTTPValidationError ^ Collapse all object <pre> detail > Expand all array<object> </pre>
LogCreate ^ Collapse all object <pre> timestamp* string date-time source_ip* string ≤ 45 characters username* string [1, 100] characters event_type > Expand all string [1, 50] characters status* string ≤ 20 characters user_agent > Expand all (string null) country > Expand all (string null) </pre>
LogOut ^ Collapse all object <pre> timestamp* string date-time source_ip* string ≤ 45 characters username* string [1, 100] characters event_type > Expand all string [1, 50] characters status* string ≤ 20 characters user_agent > Expand all (string null) country > Expand all (string null) id* integer </pre>
ValidationError ^ Collapse all object <pre> loc* > Expand all array<(string integer)> msg* string type* string </pre>

Figure 16. Detailed validation constraints for alert pages and security log payloads.

Security controls and verification

Control	Implementation	Risk addressed
Password storage	PBKDF2-SHA256 salted hash	Plaintext credential exposure
Session integrity	HMAC-signed, expiring session token	Token tampering and indefinite sessions
Cookie protection	HttpOnly, SameSite=Strict, Secure in production	Client-side theft and cross-site requests
Login throttling	Per-client failed-attempt window	Online password guessing
Secret management	Platform environment variables	Secrets committed to source control
Input validation	Typed Pydantic request schemas	Malformed or oversized input
Database transport	TLS-required managed connection	Network interception
Operational checks	Separate live and ready endpoints	Unclear deployment health

Automated verification

The project test suite completed successfully with 7 passing tests. Coverage includes API behavior, authentication and production configuration guards, plus detection behavior. Tests were executed inside the application container to keep dependencies consistent with runtime.

TEST RESULT

7 passed — API and detection checks completed successfully.

Responsible-use boundary

This project is intended for defensive monitoring, cybersecurity education, and portfolio demonstration. The events and IP addresses shown are synthetic or documentation-safe. Findings should be validated before operational action is taken.

Limitations and roadmap

Current limitations

- Detection rules are intentionally small and deterministic; they do not yet consume external threat intelligence.
- The current timeline groups historical sample events into a small number of time buckets.
- Authentication provides a single analyst identity rather than full role-based access control.
- Real-time refresh uses request-based updates rather than a persistent event stream.
- Database schema changes are not yet managed through a migration framework.

Recommended next upgrades

Priority	Upgrade	Expected benefit
P1	Alembic database migrations	Safe, repeatable production schema evolution
P1	Analyst/admin role model	Separation of duties and stronger access control
P2	WebSocket or SSE alert delivery	Near-real-time analyst notifications
P2	GeoIP and IP reputation enrichment	Higher-quality alert context
P2	Sigma-compatible rule format	Portable, maintainable detections
P3	Webhook and collector integrations	Broader log-source coverage
P3	Structured audit log	Traceable analyst and administrative actions

Portfolio case study

Problem

Authentication logs are difficult to interpret at scale, and raw failures do not automatically become investigable security findings.

Solution

I designed and deployed a mini SIEM that ingests authentication telemetry, stores it in PostgreSQL, applies detection rules, visualizes trends, and gives analysts a complete alert-triage workflow.

Result

The final project runs as a public production deployment, connects to a managed PostgreSQL database, exposes health and OpenAPI endpoints, supports bulk import and report export, and passes its automated test suite.

CV / LINKEDIN

Built and deployed a full-stack SOC monitoring dashboard using FastAPI and PostgreSQL to ingest authentication logs, detect brute-force and suspicious-location activity, visualize security metrics, and manage incident triage.

Repository and deployment

Resource	Location
Source code	github.com/bioonahnuu-design/sentinel-soc-dashboard-nahnu
Production app	sentinel-soc-dashboard-nahnu.vercel.app
API documentation	sentinel-soc-dashboard-nahnu.vercel.app/docs
Readiness check	sentinel-soc-dashboard-nahnu.vercel.app/health/ready

Author

Nahnu Rohmania
Informatics Engineering student and cybersecurity enthusiast
Universitas 17 Agustus 1945 Surabaya